Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División Ciencias de la Ingeniería
Base de Datos 2

“Análisis de Resultados”

Kenny Marco Augusto López Salazar **202031554**

Quetzaltenango 30 de octubre 2026

INFORME FINAL

En este informe presentó los componentes utilizados para cargar y transformar los datos, el análisis de los resultados obtenidos, los archivos de configuración y las conclusiones del proyecto. El sistema analiza las pruebas específicas de ingreso mediante MongoDB, Neo4j, un backend Spring Boot y una interfaz React.

1. Scripts utilizados para la carga y transformación de los datos

La carga principal no se realiza con un script independiente: está implementada en clases Java del backend y se ejecuta dentro de la aplicación Spring Boot. En la carpeta scripts_carga_transformacion adjunto copias de referencia de esas clases, conservando sus nombres originales. El código operativo permanece en backend/src/main/java/gt/usac/bd2/cargas; las copias no deben ejecutarse de manera aislada ni sustituir una versión modificada del repositorio sin revisarla.

Componentes de la carga

- CargasControlador.java: recibe el archivo mediante el endpoint de carga y expone operaciones para consultar cargas, errores, resumen y sincronización.
- CargasServicio.java: coordina la lectura del CSV, validación, inserción por lotes en MongoDB, contadores e historial de cargas y errores.
- CsvDatos.java: reconoce los encabezados, valida y normaliza los valores, conserva los originales y genera el identificador de evaluación.
- GrafoServicio.java: sincroniza las evaluaciones pendientes hacia Neo4j, crea nodos y relaciones y confirma el resultado en MongoDB.

Proceso de transformación

El archivo se lee en UTF-8 y debe contener los doce campos previstos. También se reconocen variantes de encabezados presentes en algunos archivos recientes. Cada campo se limpia de espacios; la fecha y los valores numéricos se convierten a tipos adecuados. Para el año de nacimiento sin registro se conserva la ausencia y se agrega una advertencia; no se inventa una edad.

El identificador evaluacion_id se calcula con SHA-256 a partir de los valores originales ordenados. Esto permite reconocer una fila exactamente igual en una recarga. Las filas nuevas se insertan, las repetidas se cuentan como duplicadas y las inválidas se registran con su motivo en errores_carga. Los originales y las advertencias permanecen asociados a la evaluación válida.

Después de almacenar los documentos, el sistema sincroniza los registros pendientes en lotes hacia Neo4j. Se utiliza MERGE para evitar crear nuevamente los mismos nodos y relaciones al reintentar.

Si Neo4j no está disponible, los datos confirmados permanecen en MongoDB y la sincronización puede reanudarse posteriormente.

Otros scripts del proyecto

En las capturas del repositorio se observan scripts/mongodb/01_inicializar.js y scripts/neo4j/01_restricciones.cypher, usados para preparar colecciones, índices y restricciones. El archivo scripts/mongodb/02_consultas.js sirve para consultas; no reemplaza el proceso de importación del backend. Los verificadores de fases y la auditoría utilizados durante las pruebas comprueban resultados y consistencia.

2. Análisis de resultados

Volumen y calidad de los datos

Los diez archivos originales cubren de 2015 a 2024 y contienen 208453 filas. La auditoría identificó 207619 evaluaciones únicas y 834 duplicados exactos, aproximadamente el 0.40 % de las filas. La recarga del archivo de 2024 mostró 13863 duplicados y cero nuevas inserciones, lo que comprobó que una recarga exacta no vuelve a insertar esas evaluaciones.

Entre las evaluaciones únicas se identificaron 29701 aprobadas y 177918 no aprobadas. Esto equivale aproximadamente al 14.31 % y 85.69 %, respectivamente. La unidad de análisis es la evaluación: una persona puede aparecer en más de una evaluación y una aprobación de prueba no significa por sí sola que haya completado el proceso de admisión.

Los registros sin año de nacimiento válido se conservan, pero no se deben utilizar para calcular edad. Algunos archivos recientes pueden presentar textos truncados; estos valores se mantienen como aparecen en la fuente. Los campos de municipio y departamento describen la institución educativa y no deben interpretarse automáticamente como lugar de residencia.

Consistencia entre las bases y consultas

La auditoría comparó identificadores, año y resultado en MongoDB y Neo4j frente a los CSV; también verificó las relaciones principales y cero pendientes de sincronización. La comparación pasó con 207619 evaluaciones. Esta prueba no compara todos los atributos del documento ni todos los nodos auxiliares aislados.

Con el conjunto completo se verificaron 20 consultas de MongoDB y cinco de Neo4j. El resumen por carrera se regeneró con 15 carreras. El buscador, los filtros, el historial y el detalle también pasaron la verificación con un correlativo real. Los resultados agrupados deben interpretarse según su unidad: evaluaciones, aspirantes distintos, porcentajes o grupos; no siempre es correcto sumar todos los grupos para obtener personas únicas.

Persistencia y rendimiento

Después de detener e iniciar los contenedores se repitió la auditoría y los identificadores, años, resultados y relaciones verificadas siguieron coincidiendo. El ensayo cubre un reinicio de contenedores, no demuestra por sí solo un reinicio completo del equipo ni una restauración desde respaldo.

En la medición reportada, las medianas de las consultas MongoDB fueron de 4.37 a 1168.39 ms; las de Neo4j, de 2.56 a 7.43 segundos. Se ejecutaron dos solicitudes por consulta y se midió el tiempo HTTP completo de una página, incluyendo API, consulta, conteo y serialización. Los resultados reflejan el equipo y las condiciones de la prueba; no son una prueba de usuarios concurrentes.

3. Archivos de configuración necesarios para ejecutar el proyecto

Según el repositorio mostrado, compose.yaml define los servicios de bases de datos y sus volúmenes. .env.example documenta las variables necesarias; se debe crear el archivo .env local con valores propios, sin subir credenciales reales al repositorio. Los nombres exactos y opciones de Compose deben revisarse en los archivos del repositorio antes de replicar el entorno.

- backend/pom.xml: dependencias y construcción Maven; el proyecto usa Java 21 y Spring Boot 3.5.16.
- backend/src/main/resources/application.yml: puerto de la API, conexiones MongoDB y Neo4j, autenticación por variables, límites de archivos y origen del frontend.
- backend/.mvn/wrapper/maven-wrapper.properties y backend/mvnw: configuración y ejecución de Maven Wrapper; backend/ejecutar.sh carga el .env local y ejecuta Spring Boot.
- frontend/package.json y frontend/package-lock.json: dependencias y versiones del frontend React; la configuración de desarrollo dirige las solicitudes de la API al backend.
- scripts/mongodb/01_inicializar.js y scripts/neo4j/01_restricciones.cypher: inicialización de las estructuras, índices y restricciones, cuando corresponde a una base nueva.

Los servicios observados usan MongoDB 7.0 en localhost:27018, Neo4j 5.26 en localhost:7474 y 7687, backend en localhost:8091 y frontend de desarrollo en localhost:5173. MongoDB 7.0 se utilizó por compatibilidad con el kernel del equipo donde se desarrolló el proyecto.

Inicio en el entorno preparado

```
cd ~/Documentos/ProyectoFinal_Bases2
docker compose up -d
docker compose ps

cd ~/Documentos/ProyectoFinal_Bases2/backend
./ejecutar.sh
```

```
cd ~/Documentos/ProyectoFinal_Bases2/frontend
npm ci
npm run dev
```

Las dos últimas acciones se ejecutan en terminales separadas. Una vez iniciado el sistema, el endpoint http://localhost:8091/api/v1/estado permite comprobar la disponibilidad de ambas bases. Los archivos CSV y las credenciales no se adjuntan al informe.

4. Conclusiones

- El uso conjunto de MongoDB y Neo4j permitió resolver consultas agregadas y explorar relaciones entre aspirantes, evaluaciones y procedencia institucional.
- La validación, conservación de valores originales y registro de rechazos y advertencias hicieron posible procesar la fuente sin ocultar sus limitaciones.
- La identificación de filas exactas y los reintentos de sincronización facilitaron la recarga sin duplicar evaluaciones ya procesadas.
- La auditoría confirmó coincidencia de los campos y relaciones comprobados entre los CSV y ambas bases, incluso después del reinicio de los contenedores.
- Los tiempos observados permiten identificar consultas que podrían optimizarse, especialmente en Neo4j; se requieren más mediciones para evaluar concurrencia y rendimiento en otros entornos.
- El análisis debe diferenciar evaluaciones de personas y no debe interpretar una aprobación como admisión definitiva.